DUCS5006

COMPUTATIONAL COGNITIVE SCIENCE

20 CREDIT MODULE

MODULE LEADER: Dr. Sayed Hagag.

Nouran Mohamed Sharf

Reference no.:

Table of Contents

1. Introduction 3
2. Background & Theory 3
3. Data Preparation and Modeling Pipeline 5
   3.1 Dataset description: 5
   3.2 text preprocessing: 5
   3.3 Data Exploration: 6
   3.4 Feature Representation: 7
   3.5 Feature Selection 7
   3.6 Training and prediction: 8
.4 Model Implementation from Scratch 9
   4.1 Model initialization: 9
   4.2 Estimation of class prior probability: 9
   4.3 Estimation of word likelihood: 9
   4.4 Prediction and Decision Rule: 10
5. Results 11
6. Conclusion 11
7. References 12

1. Introduction

Sentiment analysis is the task of determining whether a piece of text expresses a positive or negative opinion. It has important applications in analyzing customer feedback, social media content, and movie reviews. This report presents the implementation of a Multinomial Naive Bayes classifier built from scratch, illustrating the core mechanics of the algorithm in classifying textual sentiment and the used data was the IMDb dataset, which is a dataset for binary sentiment classification having 50,000 movie reviews labeled as positive or negative

2. Background & Theory

The Multinomial Naive Bayes classifier is a probabilistic classification algorithm based on Bayes' theorem. Given a document d and a set of classes C the classifier assigns the document to the class $\hat{c}$ with the highest posterior probability, under the simplifying assumption that features are conditionally independent given the class label.

$$\hat{c} = argmax_{\{c \in C\}} P(c|d)$$

Using Bayes' theorem the posterior probability can be expressed in terms of the class prior and the likelihood of the document given the class:

$$\overbrace{P(c \mid d)}^{posterior} = \frac{\overbrace{P(d \mid c)}^{Likelihood} \overbrace{P(c)}^{prior}}{\underbrace{P(d)}_{evidence}}$$

We can simplify Baye's rule by dropping the evidence $p(d)$ because we will be computing the posterior for each possible class so $p(d)$ does not change for each class. we are always asking about most likely class for the same document that have the same evidence $p(d)$:

$$\hat{c} = \text{argmax}_{\{c \in C\}} \text{P(c|d)} = \text{argmax}_{\{c \in C\}} \overbrace{P(d \mid c)}^{Likelihood} \overbrace{P(c)}^{prior}$$

We can represent a document d as a set of features (words) w1, w2,…, w_n the main challenge now lies in computing the likelihood term $P(d \mid c)$. Without any simplifying assumptions this likelihood represents the joint probability of all words appearing together in the document given the class:

$$P(d \mid c) = P(w1, w2, \ldots, wn \mid c)$$

It would require huge numbers of parameters and impossibly large training sets. To make the computation feasible Naive Bayes classifier depends on a simplified representation of text and a conditional independence assumption.

First, documents are represented using the Bag of words (BoW) model. We converted the text into numbers by counting word occurrences, and the position does not matter so that the model can work with it.

(For example, given the text *"I love this movie love love"* and the vocabulary $[I, love, this, movie, hate]$, BoW vector is $[1, 3, 1, 1, 0]$)

Even with the bag of words, the computation of the likelihood is complex, so to simplify the computation of the joint probability, the Naive Bayes classifier operates under the assumption of conditional independence which assumes that the words are independent, so the joint probability will be converted to the product of the probability of each word given the class to another:

$$P(w1, w2, \ldots, wn|c) = P(w1|c) \cdot P(w2|c) \cdot \ldots \cdot P(wn|c)$$

But because of the number of words in the document, the products will be numerous, so the Multinomial Naive Bayes solves this problem by using the word frequencies from the BoW representation directly, raising the probability of each word to the power of its frequency:

$$P(d|c) = \prod_{i=1}^{n} P(wi \mid c)^{fi}$$

Finally, this calculation for the language models will be done in log space to avoid underflow and increase speed.

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} \log P(c) + \sum_{i \in positions} \log P(w_i|c)$$

3. Data Preparation and Modeling Pipeline

The project performs sentiment analysis by classifying text reviews as positive or negative. The process starts with the raw text input, which is then preprocessed and represented in numerical form suitable for the model. The model is trained to estimate the probability that a given review belongs to each class and predictions are made by selecting the class with the highest probability.

3.1 Dataset description:

The used dataset was “IMDB Dataset.csv” which contains two columns first column is for the reviews which varying in length second column for the label positive or negative. it consists of 50,000 rows

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

Figure 1 sample from used dataset

3.2 text preprocessing:

Before applying the model to the text the first column “reviews” undergoes preprocessing. This includes converting all characters to lowercase , remove any HTML tags , remove stop words and everything except English letters and spaces then splits text into individual words

```python
def clean_text(text):
    text = text.lower()
    text = re.sub(r'<.*?>', ' ', text)
    text = re.sub(r'[^a-z\s]', ' ', text)
    words = text.split()
    words = [w for w in words if w not in ENGLISH_STOP_WORDS]
    return ' '.join(words)
```

Figure 3 function for text cleaning

	review	clean_text
0	One of the other reviewers has mentioned that ...	reviewers mentioned watching just oz episode l...
1	A wonderful little production. The...	wonderful little production filming technique ...
2	I thought this was a wonderful way to spend ti...	thought wonderful way spend time hot summer we...
3	Basically there's a family where a little boy ...	basically s family little boy jake thinks s zo...
4	Petter Mattei's "Love in the Time of Money" is...	petter mattei s love time money visually stunn...

Figure 2 cleaned text output

3.3 Data Exploration:

After cleaning the data, we explore the data to know how to deal with it. According to the distribution of review length (word count) graph we knew that the average word count in positive reviews was 111.89 ,average word count in negative reviews was 109.18, shortest review was 3 words, and longest review was 1338 words

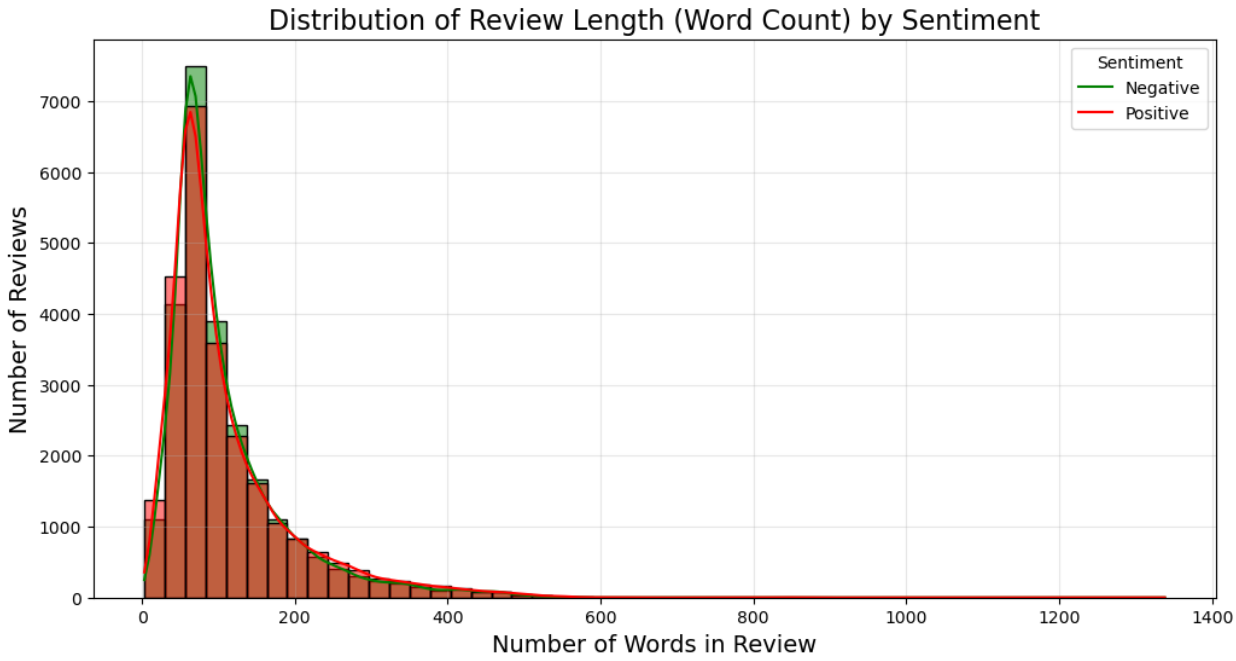


Figure 4 distribution of review length by sentiment

We also examined the most frequent words (top-20) in each sentiment class these words are less informative for sentiment classification:

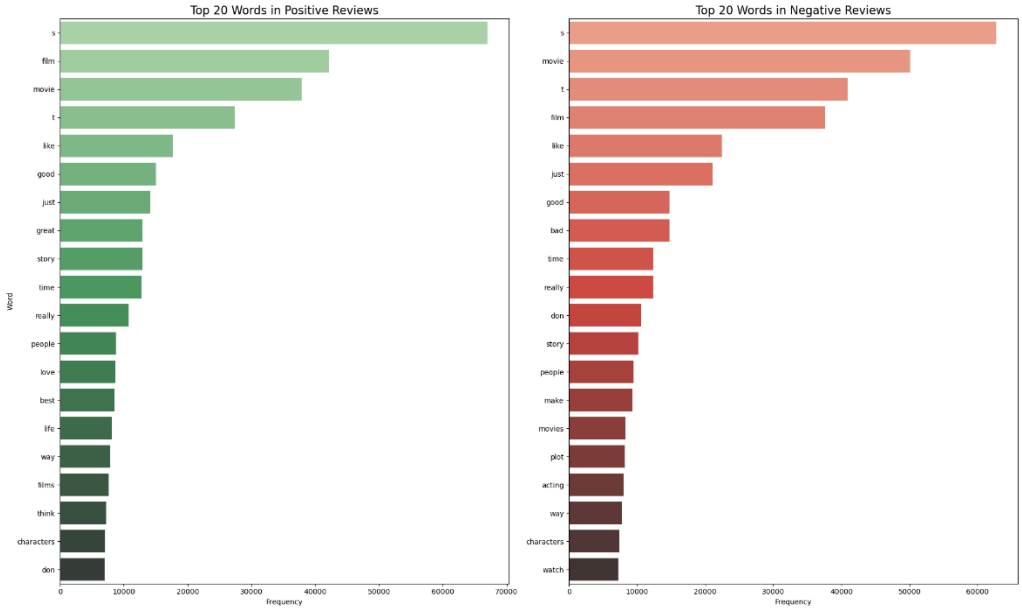


Figure 5 Top 20 most frequent words for both classes

To better understand the vocabulary and frequent words in each sentiment class, word clouds were generated for positive and negative reviews. These provide a visual overview of class specific language patterns.



Figure 6 Word Clouds by Sentiment Class

3.4 Feature Representation:

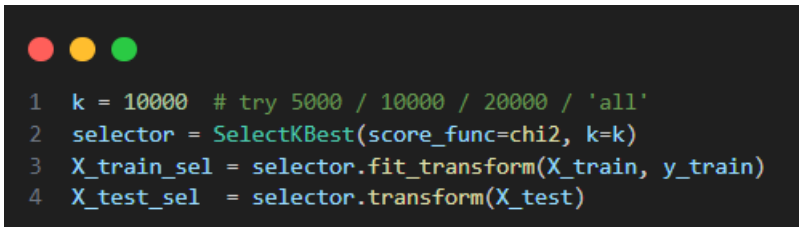
After cleaning the reviews, we convert them into numbers using the Bag of Words approach. Each review becomes a vector where each value represents how many times a word appears in that review. We use CountVectorizer with stop words removal and min/max document frequency.

```
vectorizer = CountVectorizer(
    stop_words="english",
    max_df=0.2,
    min_df=2,          # optional: ignore very rare words
    )
```

Figure 7 CountVectorizer code

3.5 Feature Selection

Not all words are equally useful for classification. To keep only the most important ones we select the top 10,000 words that are most related to the sentiment labels. This reduces the number of features the model has to process, speeds up training, and ensures it focuses on words that indicate whether a review is positive or negative.

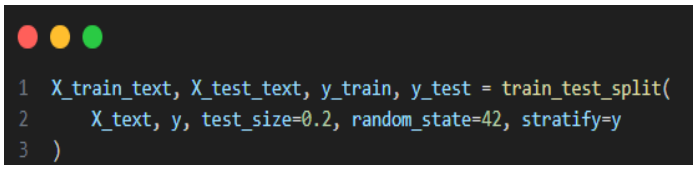


```
k = 10000  # try 5000 / 10000 / 20000 / 'all'
selector = SelectKBest(score_func=chi2, k=k)
X_train_sel = selector.fit_transform(X_train, y_train)
X_test_sel  = selector.transform(X_test)
```

Figure 8 features selected using a chi-squared statistic

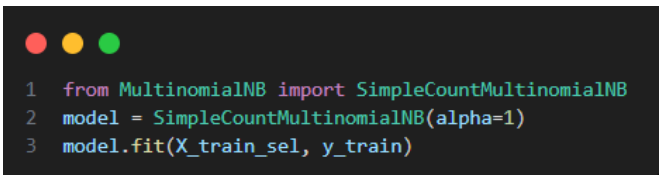
3.6 Training and prediction:

The dataset is split into train and test in ratio 80 : 20 then select the features to reduce noise and focus on the most informative words after applying counvectorizer to it. The Multinomial Naive Bayes model uses the selected features.



```
X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text, y, test_size=0.2, random_state=42, stratify=y
)
```

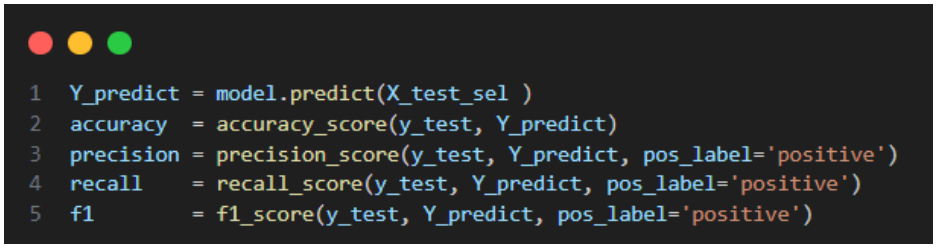
Figure 10 train-test-split



```
from MultinomialNB import SimpleCountMultinomialNB
model = SimpleCountMultinomialNB(alpha=1)
model.fit(X_train_sel, y_train)
```

Figure 9 multinomial NB train

For prediction, the same preprocessing, vectorization, and feature selection steps are applied to the test data. The trained model computes a score for each class and assigns the sentiment label with the highest score to each review.



```
Y_predict = model.predict(X_test_sel )
accuracy  = accuracy_score(y_test, Y_predict)
precision = precision_score(y_test, Y_predict, pos_label='positive')
recall    = recall_score(y_test, Y_predict, pos_label='positive')
f1        = f1_score(y_test, Y_predict, pos_label='positive')
```

Figure 11 Model prediction and performance evaluation on the test set

4. Model Implementation from Scratch

This section describes the implementation of Multinomial NB from scratch using python and NumPy , this section follows the theoretical formulation discussed in the second section. Supporting sparse input matrices

The classifier is implemented as a class named SimpleCountMultinomialNB which includes methods for model initialization, training, and prediction.

4.1 Model initialization:

It initialized through the `__init__` method with a smoothing parameter α which is used for Laplace smoothing to handle zero frequency problem, the model also defines placeholders for the learned parameters such as class labels (`self.classes_`) , log prior probabilities (`self.log_prior_`), and the log conditional probabilities of words given each class (`self.log_prob_`).

4.2 Estimation of class prior probability:

During training model calculates the probability of each class based on its frequency in the training dataset and to improve numerical stability, the log of the prior probabilities is stored in (`self.log_prior_`) and later used during prediction.

$$P(c) = \frac{Number\ of\ reviews\ in\ the\ class}{Total\ number\ of\ reviews}$$

4.3 Estimation of word likelihood:

The likelihood of a word given a class is calculated by counting how many times each word appears in reviews of each class . These counts are stored in the variable `word_count` , where each row represents a class and each column represents a word. Laplace smoothing is applied using the parameter α .After smoothing the word counts are normalized by the total number of words in each class. The final results are stored in (`self.log_prob_`) ,which contains the log probabilities of words given each class and is later used during prediction.

4.4 Prediction and Decision Rule:

After training the model, any new review is converted into the same numerical representation used during training. for each review the model calculates score for each class by summing the log likelihood of the words in the review and adding the log prior of the corresponding class.

These scores are stored in a matrix, where each row represents a review and each column represents a class.

At the end the class with the highest score is selected as the predicted label for the review, and the expected labels are returned as the model's output.

The advantage here is that this process supports sparse matrices and is done using matrix multiplication, which makes predictions faster and more efficient on large datasets instead of doing a loop for each review.

```python
import numpy as np
from scipy.sparse import issparse

class SimpleCountMultinomialNB:
    def __init__(self, alpha=1.0):
        self.alpha = alpha
        self.classes_ = None
        self.log_prior_ = None
        self.log_prob_ = None

    def fit(self, X, y):
        y = np.asarray(y)
        self.classes_, y_idx = np.unique(y, return_inverse=True)
        n_classes = len(self.classes_)
        n_features = X.shape[1]

        class_count = np.bincount(y_idx)
        self.log_prior_ = np.log(class_count) - np.log(len(y))

        word_count = np.zeros((n_classes, n_features), dtype=float)
        for c in range(n_classes):
            rows = (y_idx == c)
            if issparse(X):
                word_count[c] = np.asarray(X[rows].sum(axis=0)).ravel()
            else:
                word_count[c] = X[rows].sum(axis=0)

        smoothed = word_count + self.alpha
        denom = smoothed.sum(axis=1, keepdims=True)
        self.log_prob_ = np.log(smoothed / denom)
        return self

    def predict(self, X):
        if issparse(X):
            scores = X @ self.log_prob_.T
        else:
            scores = np.dot(X, self.log_prob_.T)
        scores = scores + self.log_prior_
        return self.classes_[scores.argmax(axis=1)]
```

Figure 12 Multinomial Naive Bayes implementation from scratch

5. Results

After training the model and applying it to the test data, the Multinomial Naive Bayes classifier performed well on the IMDB dataset.

The main performance metrics are:

- **Accuracy:** 0.8591
- **Precision:** 0.8718
- **Recall:** 0.8420
- **F1-score:** 0.8566

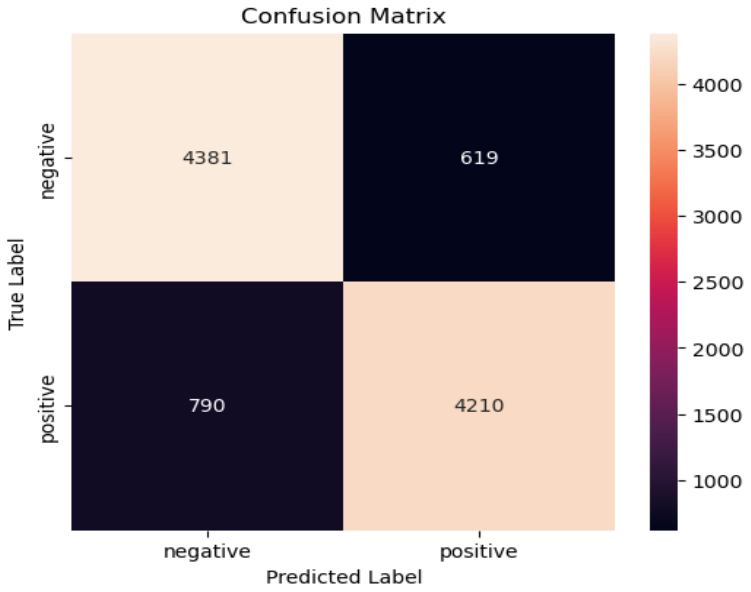


Figure 13 confusion matrix

6. Conclusion

After building and training the Multinomial NB model from scratch and applied it to classify IMDB movie reviews as positive or negative. The model performed well with accuracy of 0.8591 and balanced precision, recall and F1-scores. It was able to classify positive and negative reviews . This project shows that even a simple probabilistic model can do sentiment analysis effectively while keeping the process clear and easy to understand.

7. References

Jurafsky, D. and Martin, J.H., 2023. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. 3rd ed. [online] Available at: https://web.stanford.edu/~jurafsky/slp3/

Kaggle, 2025. *IMDB Dataset of 50K Movie Reviews*. [online] Available at: https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews?resource=download

GeeksforGeeks, 2025. *Naive Bayes Classifiers*. [online] Available at: https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/

Scikit-learn, 2025. *Naive Bayes classifiers*. [online] Available at: https://scikit-learn.org/stable/modules/naive_bayes.html